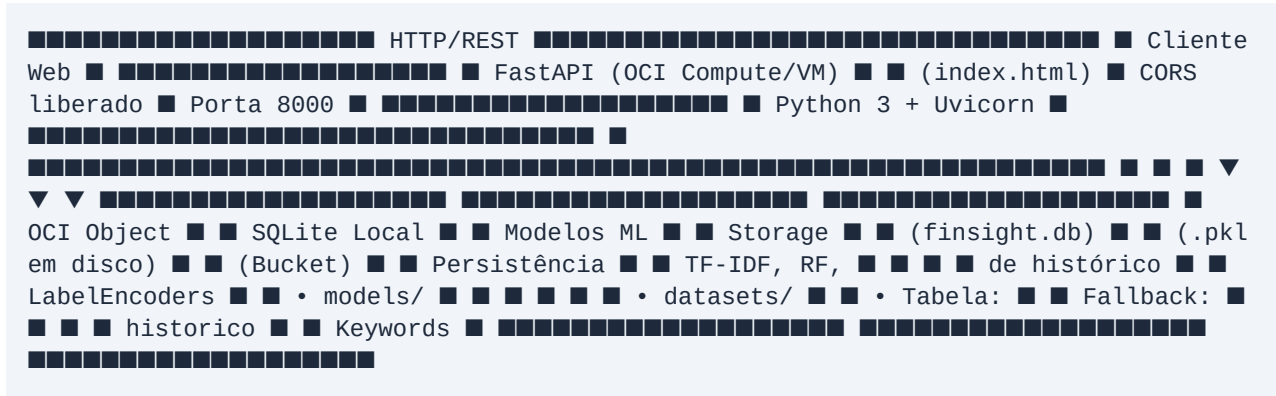


Arquitetura e Infraestrutura Oracle Cloud Infrastructure

FinSight AI v5.0 — G9-BR-TEAM-20 | Hackathon OCI 2026

1. Diagrama de Arquitetura

A arquitetura do FinSight AI segue um modelo **cloud-native** com componentes distribuídos entre a aplicação Python/FastAPI, armazenamento OCI Object Storage, banco de dados local SQLite e frontend estático.



2. Serviços OCI Utilizados

2.1 OCI Object Storage

Bucket: hackathon-one-g9-team-20

Namespace: Detectado automaticamente via API OCI

Uso: Armazenamento centralizado dos modelos de Machine Learning serializados (.pkl).

Artefatos armazenados:

Arquivo	Prefixo no Bucket	Função
vetorizador_tfidf.pkl	models/	Vetorização TF-IDF das descrições
modelo_categoria_producao.pkl	models/	Classificador de categorias (SVM/RF)
codificador_categorias.pkl	models/	LabelEncoder para categorias
modelo_perfil_producao.pkl	models/	Classificador de perfil (RF/XGBoost)
codificador_perfil.pkl	models/	LabelEncoder para perfis

Mecanismo de Sync: A aplicação tenta autenticar via Instance Principals (signer OCI). Baixa modelos do bucket se o tamanho local diferir do remoto (cache inteligente). Em falha, usa modelos locais já existentes. Retry automático: 3 tentativas por arquivo.

2.2 OCI Compute (VM)

Função: Hospedagem da aplicação FastAPI.

Porta: 8000 (Uvicorn).

Comando de inicialização: `uvicorn main:app --host 0.0.0.0 --port 8000`

Requisitos mínimos recomendados: 2 vCPUs, 4GB RAM (para carregamento dos modelos .pkl em memória).

2.3 OCI IAM (Instance Principals)

A aplicação utiliza **Instance Principals Security Token Signer** para acesso ao Object Storage sem necessidade de configuração manual de API keys. Requer que a instância Compute tenha a Dynamic Group e Policy adequadas no OCI IAM.

Nota: Policy necessária: `allow dynamic-group finsight-dg to read objects in compartment id where target.bucket.name='hackathon-one-g9-team-20'`

3. Banco de Dados — SQLite

Engine: SQLite 3 (biblioteca padrão Python)

Caminho: `~/finsight-ai/data/finsight.db`

Esquema da tabela historico:

```
CREATE TABLE historico ( id INTEGER PRIMARY KEY AUTOINCREMENT, timestamp TEXT NOT NULL, renda_mensal REAL, nivel_endividamento REAL, frequencia_poupanca TEXT, perfil_financeiro TEXT, probabilidade REAL, score_saude INTEGER, total_gasto REAL, percentual_renda_gasta REAL, comprometimento_gastos REAL, resumo_gastos TEXT, -- JSON transacoes TEXT, -- JSON recomendacoes TEXT, -- JSON alertas TEXT, -- JSON explicabilidade TEXT, -- JSON metas TEXT -- JSON );
```

Operações suportadas: INSERT (via `/analise-financeira`), SELECT (via `/historico`, `/relatorio/{id}`), DELETE (via `/historico/{id}` e `/historico/limpar`), VACUUM (pós limpeza).

Limitações: SQLite é serverless e file-based. Não suporta concorrência de escrita intensa. Para escala, migrar para OCI Autonomous Database (Oracle) ou MySQL HeatWave.

4. Caminhos e Diretórios

Diretório/Arquivo	Caminho Absoluto	Propósito
Base	<code>~/finsight-ai</code>	Diretório raiz da aplicação
Modelos ML	<code>~/finsight-ai/models/</code>	Cache local dos .pkl baixados do OCI
Dados	<code>~/finsight-ai/data/</code>	SQLite e datasets auxiliares
Banco SQLite	<code>~/finsight-ai/data/finsight.db</code>	Persistência do histórico
Backend	<code>~/finsight-ai/main.py</code>	Aplicação FastAPI principal
Frontend	<code>~/finsight-ai/index.html</code>	Interface web (single-page)
Logs	<code>stdout (systemd/journald)</code>	Logging estruturado com timestamps

5. Variáveis de Ambiente

Variável	Default	Descrição
OCI_BUCKET_NAME	hackathon-one-g9-team-20	Nome do bucket no OCI Object Storage
OCI_BUCKET_NAMESPACE	(auto-detect)	Namespace do tenant OCI

6. Segurança e Boas Práticas

- **CORS:** Em desenvolvimento, allow_origins=['*']. Em produção, restringir para o domínio do frontend.
- **Validação:** Todos os inputs passam por validação Pydantic com tipos e ranges.
- **Sanitização:** Descrições de transação são processadas em lowercase para o modelo ML.
- **Cache de Modelos:** Modelos são carregados uma vez na inicialização e mantidos em memória. Recarregamento via /reload-models.
- **GC:** Coletor de lixo executado após carregamento de modelos para prevenir OOM.
- **Retry OCI:** 3 tentativas com backoff implícito para downloads do Object Storage.

7. Escalabilidade e Limitações

Aspecto	Limite Atual	Solução de Escala Futura
Concorrência SQLite	Escrita sequencial	OCI Autonomous DB / PostgreSQL
Tamanho dos modelos	Memória RAM da VM	Modelos quantizados ou OCI Functions
Processamento CSV	Síncrono (blocking)	Fila OCI Queue + worker assíncrono
Frontend	Single-page estático	CDN OCI + React/Vue SPA
Autenticação	Nenhuma	OCI Identity + OAuth2/JWT